OurMocks

2027 CYCLE

MOCK · PRACTICE · CRACK

CUET UG

Computer Science

Python Exception & File Handling

Syntax Errors, Built-in Exceptions, File Modes, Stream Pointers & Pickle Serialization

INSIDE
Concept Notes

INSIDE
Solved Examples

INSIDE
Practice MCQs + Solutions

Aligned to the latest official syllabus · Concept-first explanations

www.ourmocks.com

Syllabus Alignment

Section / Unit	Official NTA Syllabus Pointers	Core Focus & High-Yield Exam Traps
Section B1: Topic 1 (Class 12)	Exception Handling: Syntax errors vs runtime exceptions, need of exception handling, try-except-else-finally clauses, raising exceptions, built-in exception classes.	Execution order of `else` (only when zero errors occur) vs `finally` (runs unconditionally). Hierarchy of built-in exceptions (`ZeroDivisionError`, `ValueError`, `TypeError`).
Section B1: Topic 1 (Class 12)	File Handling: Text vs Binary files, file opening modes (`r`, `w`, `a`, `r+`, `w+`, `a+`, `rb`, `wb`), reading & writing text streams.	Differences between `read()`, `readline()`, and `readlines()`. Overwrite vs append semantics, `with open(...)` auto-closing mechanics, newline stripping.
Section B1: Topic 1 (Class 12)	Stream Navigation & Binary Files: Setting offsets via `seek()`, inspecting position with `tell()`, `pickle.dump()` and `pickle.load()`.	Pointer references in `seek(offset, whence)`: `whence=0` (start), `whence=1` (current), `whence=2` (end). End-of-file detection using `try-except EOFError`.

EXAM PATTERN

CUET UG Exam Architecture: Unit 1 accounts for approximately 8% to 12% of Section B1 in the CUET UG Computer Science paper (4 to 5 MCQs). Common high-yield question archetypes include tracing `try-except-else-finally` code blocks with return statements, calculating exact character/byte counts from `read(n)`, predicting pointer offsets after multiple `seek()` and `tell()` operations, and parsing dictionary records stored via `pickle.dump()`.

Concept Module 1: Errors, Exceptions & Built-in Hierarchy

1.1 Compilation Phase vs Runtime Interpretation Phase

In the Python execution lifecycle, code undergoes two distinct phases before output is manifested:

1. **Compilation (Parsing Phase):** The Python compiler checks source code against grammar specifications. It transforms source tokens into an Abstract Syntax Tree (AST) and then into executable bytecode (`.pyc`).
   - If grammar rules are breached (e.g., missing colons after conditional statements, unbalanced parentheses or quotes, improper indentation levels), a **Syntax Error** is emitted.
   - **Key Characteristic:** Compilation happens before program execution begins. If a syntax error exists anywhere in a module, execution is aborted immediately; not even the very first line of code executes.
2. **Interpretation (Runtime Execution Phase):** The Python Virtual Machine (PVM) sequentially interprets the generated bytecode instructions.
   - If a syntactically correct instruction encounters an invalid operational condition during execution (e.g., attempting to divide a number by zero, referencing a non-existent dictionary key, converting an alphabetic string to an integer), the PVM cannot complete the instruction.
   - Instead of silently corrupting data, the PVM halts normal instruction flow and instantiates an **Exception object**. If not handled, this exception prints a traceback and terminates the program.

1.2 The Built-in Exception Hierarchy

All Python exceptions are standard object-oriented classes organized in an inheritance hierarchy rooted at `BaseException`. When an exception is raised, Python searches for an `except` clause whose specified class matches the exception class or any of its ancestor base classes.
Key standard exception classes and their operational roles:

- `BaseException`: The ultimate root of all exception classes. Directly inherits system-exiting signals:
  - `SystemExit`: Raised by `sys.exit()`. Bypasses normal `except Exception:` handlers so programs can cleanly terminate.
  - `KeyboardInterrupt`: Raised when the user hits the interrupt key combination (typically `Ctrl+C`).
- `Exception`: The root superclass for all non-system-exiting, application-level exceptions. **All user-defined exceptions must inherit from this class.**
- `ArithmeticError`: Base class for errors occurring during numeric arithmetic calculations:
  - `ZeroDivisionError`: Raised when the second argument of a division (`/`, `//`) or modulo (`%`) operation is numeric zero.
  - `OverflowError`: Raised when an arithmetic operation produces a numeric result too large to be represented (mostly in floating-point operations).
  - `FloatingPointError`: Raised when a floating-point calculation fails.
- `LookupError`: Base class for errors raised when a sequence index or dictionary key is invalid:
  - `IndexError`: Raised when a sequence subscript (list, tuple, string) is out of range.
  - `KeyError`: Raised when a mapping (dictionary) key is not found in the set of existing keys.
- `TypeError`: Raised when an operation or built-in function is applied to an object of inappropriate data type (e.g., concatenating an integer to a string: `'Score: ' + 95`).
- `ValueError`: Raised when an operation receives an argument that has the correct data type but an inappropriate value (e.g., `int('45.6')` or `math.sqrt(-9)`).
- `NameError`: Raised when an identifier (variable, function, module) is referenced that has not been bound in the local or global namespace.
  - `UnboundLocalError`: A subclass of `NameError` raised when a reference is made to a local variable in a function, but no value has been bound to that variable prior to access.
- `OSError / IOError`: Raised when an operating system operation fails:
  - `FileNotFoundError`: Raised when attempting to open a file in read mode that does not exist in the specified path.
  - `PermissionError`: Raised when attempting to access a file without appropriate filesystem read/write privileges.
  - `FileExistsError`: Raised when attempting to create a file or directory that already exists.
  - `EOFError`: Raised when an input function (`input()`, `pickle.load()`) hits the end-of-file condition without reading any data.

OurMocks

Concept Module 2: The `try-except-else-finally` Architecture

2.1 The Four-Clause Compound Statement

Python structures exception handling using four cohesive keywords:

```
try:
    # 1. Guarded Block: Code that may raise an exception
    num = int(input("Enter integer divisor: "))
    result = 1000 / num
except ValueError as ve:
    # 2. Specific Exception Handler 1
    print("Invalid numeric literal entered:", ve)
except ZeroDivisionError as zde:
    # 2. Specific Exception Handler 2
    print("Division by zero is undefined:", zde)
except Exception as gen_err:
    # 2. General Fallback Handler
    print("An unexpected error occurred:", gen_err)
else:
    # 3. Success Block: Executes ONLY if try completed with ZERO exceptions
    print("Calculation successful. Quotient =", result)
finally:
    # 4. Cleanup Block: Executes UNCONDITIONALLY in all execution paths
    print("Execution complete. Releasing resources.")
```

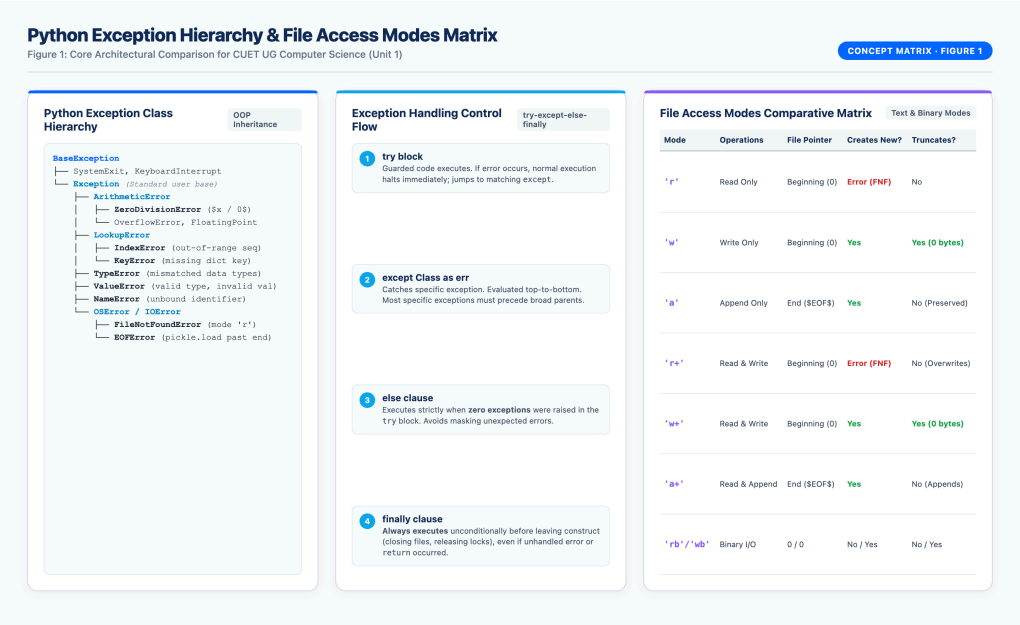


Figure 1: Figure 1: Python Built-in Exception Hierarchy & Comprehensive File Access Modes Matrix.

2.2 Execution Protocol & Path Tracing

1. **Normal Execution Path (No Exception Raised):**
   - PVM executes `try` block completely.
   - All `except` blocks are skipped entirely.
   - The `else` block executes.
   - The `finally` block executes.
2. **Caught Exception Path:**
   - PVM executes `try` block until the failing statement.
   - The remainder of the `try` block is abandoned immediately.
   - PVM scans `except` clauses top-to-bottom. The first matching class handles the exception.
   - The `else` block is **skipped**.
   - The `finally` block executes.
   - Normal program flow continues **after** the `finally` construct.
3. **Uncaught Exception Path:**
   - PVM encounters an exception in `try` with no matching `except` block.
   - The `else` block is **skipped**.
   - The `finally` block executes immediately.

- After `finally` terminates, the unhandled exception propagates up the call stack, halting the program and printing the traceback.

2.3 The `finally` Guarantee: Interactions with `return`

The `finally` block is guaranteed to execute even if the `try` or `except` blocks contain control flow altering statements such as `return`, `break`, or `continue`.

- If a function executes `return x` inside `try`, Python saves `x`, executes `finally`, and then returns `x`.
- **Fatal Trap:** If `finally` itself contains a `return y` statement, the return value `y` **silently discards and overwrites** any previous return value from `try` or `except`!

2.4 Raising Exceptions & Custom User-Defined Classes

- **The `raise` statement:** Developers can explicitly trigger exceptions using `raise ExceptionClass("message")` or re-raise an active exception using a bare `raise`:
```
def set_percentage(pct):
    if not (0 <= pct <= 100):
        raise ValueError(f"Percentage {pct} is out of valid bounds [0, 100].")
```
- **User-Defined Custom Exceptions:** Custom business logic errors are created by subclassing `Exception`:
```
class InsufficientBalanceError(Exception):
    """Raised when account balance is less than withdrawal amount."""
    def __init__(self, balance, amount):
        self.balance = balance
        self.amount = amount
        super().__init__(f"Cannot withdraw {amount}; balance is {balance}")

def process_withdrawal(bal, amt):
    if amt > bal:
        raise InsufficientBalanceError(bal, amt)
    return bal - amt
```

Concept Module 3: File Handling Fundamentals & Text Files

3.1 Operating System File Streams & Persistence

Variables stored in RAM are transient; when a script completes or the computer powers off, volatile memory is cleared. To achieve data persistence, data is serialized to secondary storage devices (SSD, HDD).

- The operating system manages physical storage sectors and exposes a high-level abstraction called a **File Stream**.
- Opening a file requests an integer **File Descriptor** from the OS kernel, which allocates internal input/output buffers in memory to optimize read/write performance.

3.2 Text Files vs Binary Files

- **Text Files (`.txt`, `.py`, `.csv`, `.tsv`):**
  - Composed of continuous sequences of ASCII or Unicode (UTF-8) characters.
  - Stored in human-readable plain text format.
  - Employs special End-Of-Line (EOL) delimiters. On Unix/macOS, EOL is a single newline character (ASCII 10). On Windows, EOL is a two-character carriage return and line feed sequence (ASCII 13 + ASCII 10).
  - **Automatic Translation:** When Python opens a text file, it automatically translates platform-specific newline characters into Python's standard during reading, and reverses this translation during writing.
- **Binary Files (`.dat`, `.bin`, `.jpg`, `.pdf`, `.mp3`):**
  - Composed of raw byte sequences ($0 - 255$) without character encoding.
  - Matches the exact internal memory representation of data structures.
  - No newline translation is performed; bytes are read and written identically without transformation.
  - Not directly readable with a standard text editor.

3.3 File Access Modes Detailed Taxonomy

When invoking `open(filename, mode)`, the `mode` string defines permissions and initial pointer location:

- `'r'` (Read Only - Default): File pointer set to byte 0 (start). File must already exist; if absent, raises `FileNotFoundError`.
- `'w'` (Write Only): File pointer set to byte 0. If file exists, **its contents are instantly truncated to 0 bytes (overwritten)**. If file does not exist, a new empty file is created.
- `'a'` (Append Only): File pointer set to the end of the file (EOF). Existing content is preserved. New data written is appended at the end. If file does not exist, a new file is created.

- `'r+'` (Read & Write): File pointer set to byte 0. Does **not** truncate existing file. Enables reading and overwriting in place. File must already exist; raises `FileNotFoundError` if missing.
- `'w+'` (Write & Read): File pointer set to byte 0. **Truncates existing file to 0 bytes immediately**. Enables reading and writing from scratch. Creates file if missing.
- `'a+'` (Append & Read): File pointer set to EOF. Preserves existing data. Writes always occur at the end regardless of current seek position. Reading is permitted by seeking back. Creates file if missing.
- Binary equivalents: Appending `'b'` (e.g., `'rb'`, `'wb'`, `'ab'`, `'rb+'`, `'wb+'`, `'ab+'`) disables text decoding and treats data as raw `bytes`.

3.4 Context Managers: The `with` Statement

Standard file handling requires explicit closure via `f.close()` to flush memory buffers and release system locks. If an exception occurs between `open()` and `close()`, the file descriptor leaks. The `with` statement implements Python's Context Management protocol (`__enter__` and `__exit__`):

```
with open("students.txt", "r", encoding="utf-8") as f:
    data = f.read()
# f is guaranteed to be closed upon exiting block scope
```

3.5 Reading Text Files: `read()`, `readline()`, and `readlines()`

1. `file.read(n=-1)`:
   - Reads at most n characters. If n is omitted or negative, reads the entire file into a single string.
   - **Memory Footprint:** Calling `read()` on massive files (e.g., 4 GB) can exhaust system RAM.
2. `file.readline(size=-1)`:
   - Reads from current position up to the next character and returns a string (including the trailing).
   - Returns an empty string `""` **strictly when the end of file is reached**. An empty line containing only a newline returns "
     ", whose boolean truthiness is `True`.
3. `file.readlines(hint=-1)`:
   - Reads all remaining lines in the file and returns them as a Python `list` of strings: `["Line 1
     ", "Line 2
     ", ...]`.
4. **Memory-Efficient Iteration (Idiomatic Python):**
   - Iterating directly over the file handle `for line in f:` uses a lazy generator buffer, reading one line into memory at a time without loading the entire file into RAM:

```
with open("data.txt", "r") as f:
    for line in f:
        print(line.strip())
```

3.6 Writing to Text Files: `write()` vs `writelines()`

1. `file.write(string)`:
   - Writes the specified string to the file at current cursor position.
   - Returns an integer indicating the number of characters written.
   - **Crucial Rule:** Does **not** automatically append . Successive `write()` calls write text adjacent to each other.
2. `file.writelines(iterable_of_strings)`:
   - Writes an iterable (list, tuple) of string elements sequentially.
   - **Crucial Rule:** Does **not** insert newline characters or spaces between elements. If newline separation is desired, elements must explicitly contain :

```
lines = ["Alpha
", "Beta
", "Gamma
"]
with open("greek.txt", "w") as f:
    f.writelines(lines)
```

Concept Module 4: Stream Pointer Manipulation (`tell` & `seek`)

4.1 The Stream Pointer Concept

The file stream pointer (cursor) maintains an offset in bytes from the start of the file (0). Every read or write operation advances the pointer by the number of bytes read or written.

4.2 `tell()` Method

The `tell()` method returns an integer representing the exact current byte position of the file pointer:

```
with open("test.txt", "w") as f:
    f.write("Hello")
    print(f.tell())  # Output: 5
```

4.3 `seek(offset, whence=0)` Method

The `seek()` method moves the file pointer to a designated location:

- `offset`: Number of bytes to shift.
- `whence`: Reference origin for calculation:
  - `0` (`SEEK_SET`): Relative to the beginning of the file. `offset` must be ≥ 0.
  - `1` (`SEEK_CUR`): Relative to the current pointer position.
  - `2` (`SEEK_END`): Relative to the end of the file. Typically uses negative offset.

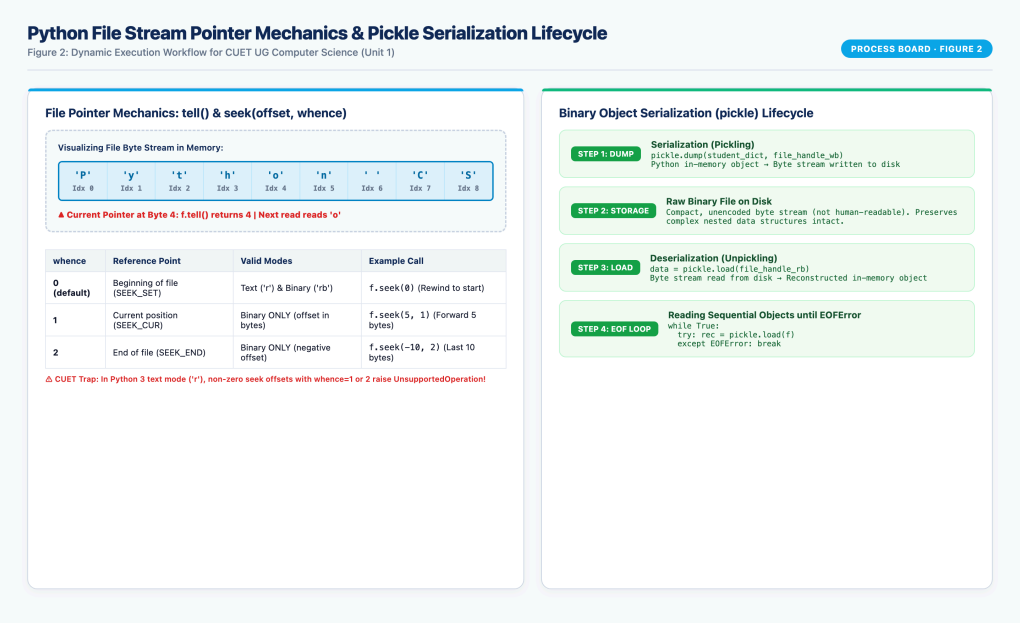


Figure 2: Figure 2: File Stream Pointer Mechanics & Pickle Binary Object Lifecycle.

4.4 Python 3 Text Mode Seek Restrictions

In Python 3, text mode decodes bytes into Unicode code points. Because variable-length encodings (such as UTF-8) use anywhere from 1 to 4 bytes per character, computing arbitrary character positions from the current pointer or the end without parsing every preceding byte is computationally intractable.

- **Strict Rule:** In text mode (`'r'`, `'w'`, `'a'`), the only valid calls to `seek()` are:
  1. `seek(n, 0)`: Jump to byte offset n from the start.
  2. `seek(0, 0)`: Rewind to the beginning.
  3. `seek(0, 2)`: Jump to the end of the file.
- Calling `seek(offset, 1)` or `seek(offset, 2)` with any non-zero offset in text mode raises `io.UnsupportedOperation: can't do nonzero cur-relative seeks`!
- To perform arbitrary seeking relative to current position (`whence=1`) or end of file (`whence=2`), the file **must** be opened in binary mode (`'rb'`, `'wb'`).

Concept Module 5: Binary File Operations with `pickle`

5.1 Object Serialization (Pickling)

Text files can only store strings. When persisting Python data structures (lists of dictionaries, tuples, custom objects), converting them to strings via `str()` loses their underlying type information (e.g., numbers become strings, integers in lists require parsing).

- **Pickling (Serialization):** The process whereby Python objects are converted into a byte stream suitable for disk storage or network transmission.

- **Unpickling (Deserialization):** The reverse process whereby byte streams are converted back into identical in-memory Python objects with original types preserved.

5.2 Core `pickle` Functions

Python includes the standard module `pickle`:

1. `pickle.dump(obj, file, protocol=None)`:
   - Serializes the Python object `obj` and writes it into the open file `file`.
   - The file handle **must be opened in binary write/append mode** (`'wb'`, `'ab'`).

```
import pickle
student = {"roll_no": 12, "name": "Deepika", "marks": 98.0}
with open("student.dat", "wb") as f:
    pickle.dump(student, f)
```

2. `pickle.load(file)`:
   - Reads the next serialized object hierarchy from the open binary file and returns the reconstructed Python object.
   - The file handle **must be opened in binary read mode** (`'rb'`).

```
import pickle
with open("student.dat", "rb") as f:
    data = pickle.load(f)
    print(type(data), data["name"])  # Output: <class 'dict'> Deepika
```

5.3 Sequential Multi-Record Processing & `EOFError`

When multiple records are written to a binary file using repeated `pickle.dump()` calls, each record is appended as an independent serialized block. When reading back records using `pickle.load()`:

- Each invocation of `pickle.load()` reads exactly one complete object and advances the file cursor to the start of the next object.
- When all objects have been consumed and the cursor reaches the end of the file, `pickle.load()` raises an `EOFError`.
- **Canonical Multi-Record Reading Pattern:**

```
import pickle
records = []
try:
    with open("students.dat", "rb") as f:
        while True:
            records.append(pickle.load(f))
except EOFError:
    # Clean termination upon reading all sequential records
    pass
except FileNotFoundError:
    print("Database file does not exist.")
```

5.4 Binary File Record Searching, Updating & Deleting

In enterprise Class 12 CS applications, binary files operate as mini-databases. Because pickled objects have variable byte lengths, updating or deleting an existing record in-place is hazardous (writing a larger record will overwrite adjacent records). The standard industrial algorithm uses the **Two-File Staging Pattern**:

1. Open the original file in `'rb'` mode and a temporary file (`temp.dat`) in `'wb'` mode.
2. Read records sequentially using `pickle.load()` inside a `try-except EOFError` loop.
3. For searching/display: Compare key fields (e.g., `record['roll_no'] == target`).
4. For updating: Modify the target record's fields in memory and write the updated record to `temp.dat`; write all non-matching records unchanged.
5. For deleting: Write all records **except** the target record to `temp.dat`.
6. Close both files.
7. Use the `os` module: `os.remove("original.dat")` followed by `os.rename("temp.dat", "original.dat")`.

5.5 Comprehensive Worked Examples

WORKED EXAMPLE 1: MULTI-CLAUSE TRACEBACK ANALYSIS

Determine the exact console output of executing the following Python script:

```
def process_values(lst, idx, divisor):
    try:
        val = lst[idx]
        quotient = val / divisor
        return quotient
    except IndexError:
        print("ERR_INDEX", end=" ")
        return -1
    except ZeroDivisionError:
```

```
        print("ERR_DIV", end=" ")
        return -2
    else:
        print("SUCCESS", end=" ")
    finally:
        print("DONE", end=" ")

print("Result 1:", process_values([10, 20, 30], 1, 5))
print("Result 2:", process_values([10, 20, 30], 5, 2))
print("Result 3:", process_values([10, 20, 30], 0, 0))
```

Solution Step-by-Step:

1. `Call 1 (idx=1, divisor=5): lst[1]=20, 20/5 = 4.0`. No exceptions raised. `else` executes, printing `"SUCCESS "`. Before returning `4.0`, `finally` executes, printing `"DONE "`. The function returns `4.0`. Output: `SUCCESS DONE Result 1: 4.0`.
2. `Call 2 (idx=5, divisor=2): lst[5]` raises `IndexError`. Caught by `except IndexError`, printing `"ERR_INDEX "`. Return value `-1` is primed. `else` is skipped. `finally` executes, printing `"DONE "`. Function returns `-1`. Output: `ERR_INDEX DONE Result 2: -1`.
3. `Call 3 (idx=0, divisor=0): lst[0]=10, 10/0` raises `ZeroDivisionError`. Caught by `except ZeroDivisionError`, printing `"ERR_DIV "`. Primed return `-2`. `else` skipped. `finally` executes, printing `"DONE "`. Function returns `-2`. Output: `ERR_DIV DONE Result 3: -2`.

WORKED EXAMPLE 2: STREAM POINTER OFFSET ARITHMETIC

A text file `log.txt` contains the following string:
`PYTHON-CUET-2027`
Trace the value printed by each `tell()` and `read()` statement:

```
with open("log.txt", "r") as f:
    print("P1:", f.tell())
    c1 = f.read(6)
    print("C1:", c1)
    print("P2:", f.tell())
    f.seek(7)
    print("P3:", f.tell())
    c2 = f.read(4)
    print("C2:", c2)
    f.seek(0)
    print("P4:", f.tell())
```

Solution Step-by-Step:

- Initial open: pointer at byte 0 → `P1: 0`.
- `read(6)` reads 6 characters (`"PYTHON"`), advancing cursor by 6 → `C1: PYTHON`, `P2: 6`.
- `seek(7)` jumps cursor to byte 7 (character index 7 is `'C'`) → `P3: 7`.
- `read(4)` reads 4 characters (`"CUET"`), advancing cursor to byte 11 → `C2: CUET`.
- `seek(0)` rewinds cursor back to the start → `P4: 0`.

WORKED EXAMPLE 3: BINARY DATABASE RECORD FILTER

Write a Python function `count_high_scorers(filename, cutoff)` that reads a binary file containing pickled dictionaries formatted as `{"roll": int, "name": str, "marks": float}` and returns the count of students scoring $\geq$ cutoff.
Solution:

```
import pickle

def count_high_scorers(filename, cutoff):
    count = 0
    try:
        with open(filename, "rb") as f:
            while True:
                student = pickle.load(f)
                if student.get("marks", 0) >= cutoff:
                    count += 1
    except FileNotFoundError:
        print(f"Error: Database file '{filename}' not found.")
        return 0
    except EOFError:
        # Normal completion when all records are processed
        pass
    return count
```

5.6 Operating System Interface & File Path Manipulation

Robust production file processing requires interacting with the underlying file system. Python supplies these operating system primitives through the standard `os` and `os.path` modules.

1. Core os File and Directory Operations

- `os.rename(src, dst)`: Renames file or directory `src` to `dst`. If `dst` already exists on Windows, `FileExistsError` is raised; on POSIX systems, it may silently overwrite if permissions permit.
- `os.remove(path)` / `os.unlink(path)`: Deletes the specified file. Raises `FileNotFoundError` if the path is invalid, and raises `IsADirectoryError` or `PermissionError` if `path` points to a directory.
- `os.rmdir(path)`: Removes an **empty** directory. Raises `OSError` (Directory not empty) if files reside within.
- `os.mkdir(path)` / `os.makedirs(path, exist_ok=True)`: Creates a single directory or an entire nested directory tree.
- `os.getcwd()`: Returns the Current Working Directory as an absolute path string.
- `os.chdir(path)`: Changes the Current Working Directory of the executing Python process.

2. Path Inspection with os.path

- `os.path.exists(path)`: Returns `True` if `path` refers to an existing file system path or directory.
- `os.path.isfile(path)`: Returns `True` strictly if `path` is an existing regular file.
- `os.path.isdir(path)`: Returns `True` strictly if `path` is an existing directory.
- `os.path.getsize(path)`: Returns the exact size of the file in bytes **without** opening the stream.
- `os.path.split(path)`: Deconstructs a path into a 2-tuple `(head, tail)`, where `tail` is the last pathname component and `head` is everything leading up to it.
- `os.path.splitext(path)`: Splits a path into `(root, ext)`. The extension `ext` contains the leading period (e.g., `os.path.splitext("data.backup.csv")` yields `("data.backup", ".csv")`).
- `os.path.join(path1, *paths)`: Concatenates path components intelligently using the host OS separator (`/` on POSIX, `\\` on Windows).

5.7 Low-Level File Buffering, Flush Protocols & OS Synchronization

To minimize expensive operating system disk I/O interrupts, Python mediates file streams through memory buffers. Understanding buffer flushing is essential for real-time logging and crash-resilient pipelines.

1. Buffering Policies in open()

The optional integer argument `buffering` controls the stream caching strategy:

- `buffering = 0`: **Unbuffered mode.** Only permitted in binary mode (`"wb"`, `"rb"`). Every read/write issues an immediate system call. Passing `0` in text mode immediately raises `ValueError`.
- `buffering = 1`: **Line-buffered mode.** Permitted only in text mode. The buffer automatically flushes whenever a newline character (`\n`) is written to the stream.
- `buffering > 1`: **Fixed chunk buffering.** Allocates an internal memory buffer of exactly the specified integer size in bytes (e.g., `buffering = 4096`).
- `buffering = -1` (Default): Uses system default buffer size, typically defined by `io.DEFAULT_BUFFER_SIZE` (8192 bytes / 8 KB on modern architectures).

2. flush() vs close() vs os.fsync()

- `f.flush()`: Clears the internal Python software buffer and writes all pending data into the operating system kernel cache. The file descriptor remains **open** and active.
- `f.close()`: Automatically calls `flush()` first, then terminates the OS file descriptor, freeing system handles. Accessing `f` after `close()` raises `ValueError: I/O operation on closed file`.
- `os.fsync(f.fileno())`: Forces the operating system kernel buffer to physically commit changes down to the non-volatile hardware disk.

5.8 Unicode Architecture, Character Sets & Encoding in Python 3

Python 3 enforces a strict distinction between textual abstractions and concrete raw binary bytes:

- `str`: Represents textual data as an immutable sequence of abstract Unicode code points. Characters have identity independent of byte representation.
- `bytes`: Represents an immutable sequence of raw 8-bit octets (integers in the closed range $[0, 255]$).

1. Encoding and Decoding Symmetry

- `str.encode(encoding="utf-8")` converts Unicode text into raw `bytes`.
- `bytes.decode(encoding="utf-8")` translates raw octets back into a Unicode `str`.

2. The encoding Parameter and Cross-Platform Portability

When opening text files via `open("file.txt", "r")` without specifying `encoding`, Python falls back to `locale.getpreferredencoding(False)`. On Windows, this frequently resolves to `cp1252` or `mbcs`, whereas Linux and macOS resolve to `utf-8`.

- If a file saved in UTF-8 containing multilingual characters (e.g. Hindi or special math symbols) is read on Windows with default settings, Python raises `UnicodeDecodeError`.
- **Golden Rule:** Always specify `encoding="utf-8"` explicitly in every text `open()` call to guarantee cross-platform deterministic behavior.

3. Error Handling Handlers in open()

When encoding or decoding encounters unmapped byte sequences, the `errors` parameter dictates behavior:

- `errors="strict"` (Default): Immediately raises `UnicodeDecodeError` or `UnicodeEncodeError`.
- `errors="ignore"`: Silently discards invalid characters/bytes, risking data loss.
- `errors="replace"`: Replaces malformed sequences with the official Unicode replacement character `\ufffd` (rendered as `?` in text).
- `errors="backslashreplace"`: Replaces malformed characters with Python backslashed escape sequences.

WORKED EXAMPLE 4: ADVANCED CSV DIALECT REGISTRATION & PARSING

A scientific laboratory stores sensor logs formatted with pipe (`|`) delimiters and hash (`#`) comment lines. Write a Python snippet registering a custom CSV dialect to parse this file cleanly:

```
import csv

# Register custom dialect
csv.register_dialect(
    "sensor_dialect",
    delimiter="|",
    quoting=csv.QUOTE_NONNUMERIC,
    skipinitialspace=True
)

data = [
    ["Sensor_ID", "Voltage", "Status_Code"],
    ["SNS-01", 3.32, 101],
    ["SNS-02", 5.01, 102]
]

with open("sensors.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f, dialect="sensor_dialect")
    writer.writerows(data)
```

Analysis: `QUOTE_NONNUMERIC` automatically encloses all non-numeric strings in quotes while outputting float and integer values as raw numbers, enabling type-preserving roundtrip parsing.

WORKED EXAMPLE 5: DIRECT BINARY RECORD SEARCH & IN-PLACE UPDATE

A binary file `accounts.dat` stores student balance records serialized as dictionaries `{"adm_no": int, "balance": float}` using `pickle`. Implement `update_student_balance(target_adm, delta)` that updates the balance without overwriting unrelated records:

```
import pickle, os

def update_student_balance(target_adm, delta):
    found = False
    temp_file = "temp_acc.dat"
    with open("accounts.dat", "rb") as fin, open(temp_file, "wb") as fout:
        while True:
            try:
                rec = pickle.load(fin)
                if rec["adm_no"] == target_adm:
                    rec["balance"] += delta
                    found = True
                pickle.dump(rec, fout)
            except EOFError:
                break
    os.remove("accounts.dat")
    os.rename(temp_file, "accounts.dat")
    return found
```

WORKED EXAMPLE 6: ENTERPRISE USER-DEFINED EXCEPTION HIERARCHY

Define a custom exception hierarchy for an automated banking system and demonstrate explicit exception chaining:

```
class BankingError(Exception):
    """Base exception for all domain errors."""
    pass

class InsufficientFundsError(BankingError):
    def __init__(self, account_id, balance, required):
        super().__init__(f"Account {account_id}: Balance {balance} < Required {required}")
        self.account_id = account_id
        self.balance = balance
        self.required = required

def withdraw(acc_id, balance, amount):
    try:
        if amount > balance:
            raise InsufficientFundsError(acc_id, balance, amount)
        return balance - amount
    except InsufficientFundsError as err:
        # Explicit exception chaining with from
        raise RuntimeError("Transaction processing failed") from err
```

Master Mindmap: Python Exception & File Handling



Figure 3: Master Synthesis Mindmap: Structural framework of Unit 1 Python Exception & File Handling.

OurMocks

High-Yield Cheatsheet & Exam Traps

MASTER PYTHON EXCEPTION & FILE HANDLING REVISION CHEATSHEET

- **Syntax Errors vs Exceptions:** Syntax errors caught at parsing (zero code runs); exceptions caught at runtime during execution.
- **Exception Hierarchy:** Root `BaseException` → `Exception` → `ArithmeticError`, `LookupError`, `TypeError`, `ValueError`, `OSError`.
- **try-except-else-finally:** `else` executes **only** when 0 exceptions occurred; `finally` executes **unconditionally** in all paths.
- **File Modes:** `'r'` (read only, pointer 0, error if missing); `'w'` (write only, pointer 0, truncates file to 0); `'a'` (append, pointer at EOF).
- **Update Modes:** `'r+'` (read/write, pointer 0, no truncation); `'w+'` (read/write, pointer 0, truncates to 0); `'a+'` (read/append, pointer at EOF).
- **Reading Text:** `read([n])` (str of length n or whole file); `readline()` (single line with); `readlines()` (list of line strings).
- **Writing Text:** `write(str)` (returns char count, no); `writelines(list)` (writes sequence, does **not** add between items).
- **Stream Pointer:** `tell()` returns current byte offset; `seek(offset, whence)` moves pointer. `whence`: 0 (start), 1 (current), 2 (end).
- **Text Seek Restriction:** In Python 3 text mode, non-zero offsets with `whence=1` or `2` raise `io.UnsupportedOperation`.
- **Pickle Serialization:** `pickle.dump(obj, f)` (writes binary stream); `pickle.load(f)` (reads reconstructed object). Files must use `'wb'` / `'rb'`.
- **Pickle Loop Termination:** Sequential `pickle.load()` raises `EOFError` when file ends; intercept with `except EOFError: break`.

THE SEVEN FATAL CUET TRAPS IN PYTHON EXCEPTION & FILE HANDLING

1. **The Overwrite vs Append Trap:** Opening a file with `'w'` or `'w+'` **immediately wipes out all existing contents** (truncates file to 0 bytes) the moment `open()` is called! Use `'a'` or `'a+'` to preserve existing data.
2. **The `else` vs `finally` Execution Trap:** The `else` block executes **only** if the `try` block completed without any exceptions. The `finally` block executes **always**, even if an exception was raised, unhandled, or if a `return` was encountered!
3. **The Missing in `writelines()` Trap:** `file.writelines(['Hello', 'World'])` does **not** write two separate lines! It concatenates them as `'HelloWorld'`. You must explicitly supply inside the strings.
4. **The `seek()` Text Mode whence Trap:** In text mode, `seek(5, 1)` or `seek(-2, 2)` raises `io.UnsupportedOperation`! Non-zero offsets with `whence=1` or `whence=2` are **strictly permitted only in binary mode** (`'rb'`).
5. **The `read()` vs `readline()` vs `readlines()` Type Trap:** `read()` returns a single `str`; `readline()` returns a single `str` ending with ; `readlines()` returns a `list` of strings. Passing `readlines()` to a string method raises `AttributeError`.
6. **Exception Specificity Ordering Trap:** Placing `except Exception:` **before** `except ValueError:` intercepts all errors at the parent level, making the specific handler dead, unreachable code. Most specific handlers must precede general ones.
7. **The Binary File Pickle String Mode Trap:** Attempting `pickle.dump(obj, f)` on a file opened with mode `'w'` (text mode) raises `TypeError: write() argument must be bytes, not str`. Binary modes (`'wb'`, `'rb'`) are mandatory.

CUET UG Exam-Style Practice Questions (15 MCQs)

Q1. Which of the following code snippets correctly handles a missing file without crashing, printing a custom warning message?

(A)
```
try:
    f = open("data.txt", "r")
except FileNotFoundError:
    print("File missing!")
```

(B)
```
try:
    f = open("data.txt", "w")
except FileNotFoundError:
    print("File missing!")
```

(C)
```
try:
    f = open("data.txt", "r")
finally:
    print("File missing!")
```

(D)
```
try:
    f = open("data.txt", "a")
except IOError:
    print("File missing!")
```

Sol. Q1. Opening a non-existent file in read mode (`'r'`) raises `FileNotFoundError`. The `try-except FileNotFoundError` block intercepts the exception and prints the custom warning. Modes `'w'` and `'a'` create the file if it does not exist, so they never raise `FileNotFoundError`.

Answer: (A)

Q2. Consider the following Python code execution:
```
def test():
    try:
        return 1
    finally:
        return 2
print(test())
```
What is printed to the console?

(A) 1
(B) 2
(C) 1n2
(D) SyntaxError

Sol. Q2. When a `return` statement is encountered inside a `try` block, the `finally` block is guaranteed to execute before the function actually exits. If the `finally` block itself executes a `return` statement, its return value overrides the previous return value. Thus, 2 is returned.

Answer: (B)

Q3. What is the effect of opening an existing text file named `notes.txt` containing 500 characters using the mode `'w+'`?

(A) The file pointer is placed at the end for appending
(B) The file is opened for reading and writing without modifying existing text
(C) The existing content is immediately erased (file size becomes 0 bytes)
(D) A FileExistsError is raised

Sol. Q3. Both `'w'` and `'w+'` modes truncate the file to zero length upon opening. Although `'w+'` enables both reading and writing, any pre-existing content is irrevocably wiped out the instant the file is opened.

Answer: (C)

Q4. Which built-in Python exception class is raised when evaluating the expression `int("12.5")`?

(A) TypeError
(B) NameError
(C) ArithmeticError
(D) ValueError

Sol. Q4. The `int()` constructor can parse integer strings (e.g., `"12"`), but a string representation of a floating-point literal (`"12.5"`) is an invalid literal for integer radix 10, raising a `ValueError`.

Answer: (D)

Q5. What does the `file.tell()` method return in Python?

(A) The current byte position of the file pointer from the beginning of the file
(B) The total number of characters remaining in the file
(C) The total line count of the file
(D) A boolean indicating whether the file is open

Sol. Q5. `tell()` returns an integer specifying the exact current position of the file cursor measured in bytes from the start of the file (0).

Answer: (A)

CUET UG Exam-Style Practice Questions (Part 2)

Q6. Consider the following code:

```
with open("data.txt", "w") as f:
    f.writelines(["CUET", "NTA", "2027"])
with open("data.txt", "r") as f:
    s = f.read()
print(s)
```

What is the exact output displayed?

(A) CUET n NTA n 2027
(B) CUETNTA2027
(C) ['CUET', 'NTA', '2027']
(D) TypeError: writelines takes string

Sol. Q6. `writelines()` writes each string in the iterable sequentially without adding newline delimiters or spaces. Thus `"CUET"`, `"NTA"`, and `"2027"` are written contiguously, resulting in `"CUETNTA2027"`.
Answer: (B)

Q7. In a binary file opened in `'rb'` mode, which function call relocates the file cursor to exactly 10 bytes before the end of the file?

(A) `f.seek(10, 0)`
(B) `f.seek(10, 2)`
(C) `f.seek(-10, 2)`
(D) `f.seek(-10, 1)`

Sol. Q7. In `seek(offset, whence)`, `whence=2` designates the end of the file (`SEEK_END`). A negative offset `-10` moves the cursor 10 bytes backward from the end. Thus `f.seek(-10, 2)` correctly positions the pointer.
Answer: (C)

Q8. Which clause in a `try-except` structure executes only if NO exceptions were raised in the `try` block?

(A) `catch`
(B) `finally`
(C) `pass`
(D) `else`

Sol. Q8. The `else` block executes strictly when the `try` block completes successfully without raising any exceptions. The `finally` block, by contrast, executes unconditionally regardless of errors.
Answer: (D)

Q9. Which module and function combination is used to serialize a Python dictionary object into a binary disk file?

(A) `pickle.dump(obj, file)`
(B) `pickle.load(obj, file)`
(C) `binary.write(obj, file)`
(D) `pickle.serialize(obj, file)`

Sol. Q9. The `pickle` module uses `dump(obj, file)` to serialize an in-memory object and write it into a binary file. The `load(file)` function performs the reverse (deserialization).
Answer: (A)

Q10. What exception is raised when calling `f.seek(5, 1)` on a file opened with mode `open("sample.txt", "r")` in Python 3?

(A) ValueError
(B) io.UnsupportedOperation
(C) PermissionError
(D) ZeroDivisionError

Sol. Q10. In Python 3 text mode (`'r'`), seeking with non-zero offsets relative to current position (`whence=1`) or end (`whence=2`) is prohibited because multi-byte UTF-8 character boundaries cannot be computed without binary decoding. It raises `io.UnsupportedOperation`.
Answer: (B)

CUET UG Exam-Style Practice Questions (Part 3)

Q11. Consider the following code snippet:

```
import pickle
records = [{"id": 1}, {"id": 2}]
with open("test.dat", "wb") as f:
    for r in records:
        pickle.dump(r, f)

with open("test.dat", "rb") as f:
    count = 0
    try:
        while True:
            rec = pickle.load(f)
            count += 1
    except EOFError:
        pass
print(count)
```

What is printed?

(A) 0
(B) 1
(C) 2
(D) Raises unhandled EOFError

Sol. Q11. Two separate dictionary objects are pickled into `test.dat`. During reading, `pickle.load()` succeeds twice, incrementing `count` to 2. On the third read attempt, end of file is reached and `EOFError` is raised, caught cleanly by `except EOFError: pass`. Output is 2.

Answer: (C)

Q12. A user-defined custom exception class in Python must directly or indirectly inherit from which base class?

(A) BaseError
(B) RuntimeError
(C) SystemError
(D) Exception

Sol. Q12. By Python standard conventions, all custom user-defined application exceptions should subclass `Exception` (or a subclass thereof) rather than `BaseException` directly, ensuring they can be caught by standard `except Exception:` handlers.

Answer: (D)

Q13. What is the output of the following code if `sample.txt` contains `"Computer Science"`?

```
with open("sample.txt", "r") as f:
    d1 = f.read(8)
    f.seek(0)
    d2 = f.read(8)
print(d1 == d2)
```

(A) True
(B) False
(C) None
(D) Raises AttributeError

Sol. Q13. `d1 = f.read(8)` reads the first 8 characters (`"Computer"`). `f.seek(0)` rewinds the file cursor back to the beginning of the file (byte 0). `d2 = f.read(8)` again reads the first 8 characters (`"Computer"`). Since both strings are identical, `d1 == d2` evaluates to `True`.

Answer: (A)

Q14. Which of the following built-in exception classes is a subclass of `LookupError`?

(A) ValueError
(B) IndexError
(C) ZeroDivisionError
(D) FileNotFoundError

Sol. Q14. `LookupError` is the base class for exceptions raised when a key or index used on a mapping or sequence is invalid. Its two primary subclasses are `IndexError` (for sequences) and `KeyError` (for dictionaries).

Answer: (B)

Q15. What is the primary operational difference between file modes `'a'` and `'a+'`?

(A) `'a'` creates the file if missing; `'a+'` raises FileNotFoundError
(B) `'a'` allows reading and writing; `'a+'` allows appending only
(C) `'a'` allows writing (appending) only; `'a+'` permits both reading and appending
(D) `'a'` truncates the file; `'a+'` preserves contents

Sol. Q15. The `'a'` mode opens a file for writing only, placing the pointer at the end of the file. The `'a+'` mode opens the file for both reading and appending. Neither mode truncates existing contents, and both create the file if it does not already exist.

Answer: (C)

OurMocks